ResumeIQ

AI-Powered Resume Screener

Semantic NLP matching that understands meaning, not just keywords.

Python 3.10+ Streamlit HuggingFace scikit-learn

What It Does

ResumeIQ is an AI-powered resume screening tool that compares resumes against job descriptions using **semantic similarity**, not keyword matching.

Traditional screening tools fail when a resume says *"ML Engineer"* but the job says *"Machine Learning Developer"*. ResumeIQ understands they mean the same thing.

Upload a resume → Paste a job description → Get an instant match score with skill analysis.

Features

- **PDF Resume Parsing** - Extracts and cleans text from any PDF resume
- **Semantic Matching** - Uses transformer embeddings to understand meaning beyond keywords
- **Match Score** - Percentage score with Strong / Moderate / Weak classification
- **Skill Gap Analysis** - Shows exactly which skills match and which are missing
- **Multi-Resume Ranking** - Upload multiple resumes and rank candidates instantly
- **Modern Dark UI** - Clean, professional interface built with Streamlit

Tech Stack

Layer	Technology
Language	Python 3.10+
NLP Model	sentence-transformers/all-MiniLM-L6-v2
Similarity	Cosine Similarity (scikit-learn)
PDF Parsing	PyPDF2
Frontend	Streamlit
Embeddings	Hugging Face Transformers

How to Run Locally

1. Clone the repository

```
git clone https://github.com/BrundaVenkatesh01/resume-screening-ai.git
cd resume-screening-ai
```

2. Create a virtual environment

```
python -m venv venv

# On Mac/Linux:
source venv/bin/activate

# On Windows:
venv\Scripts\activate
```

3. Install dependencies

```
pip install -r requirements.txt
```

4. Run the app

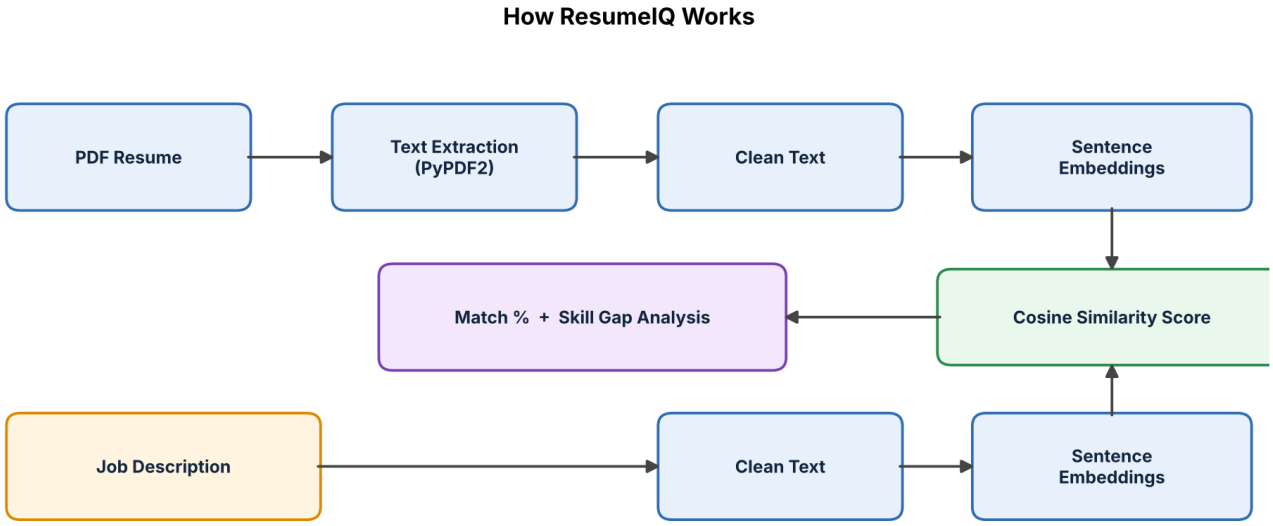
```
streamlit run app.py
```

Open your browser at `http://localhost:8501`

Project Structure

```
resume-screening-ai/
|
|-- app.py              # Streamlit web app & UI
|-- matcher.py          # NLP matching logic (embeddings + cosine similarity)
|-- resume_parser.py    # PDF text extraction & cleaning
|-- requirements.txt    # Project dependencies
`-- README.md
```

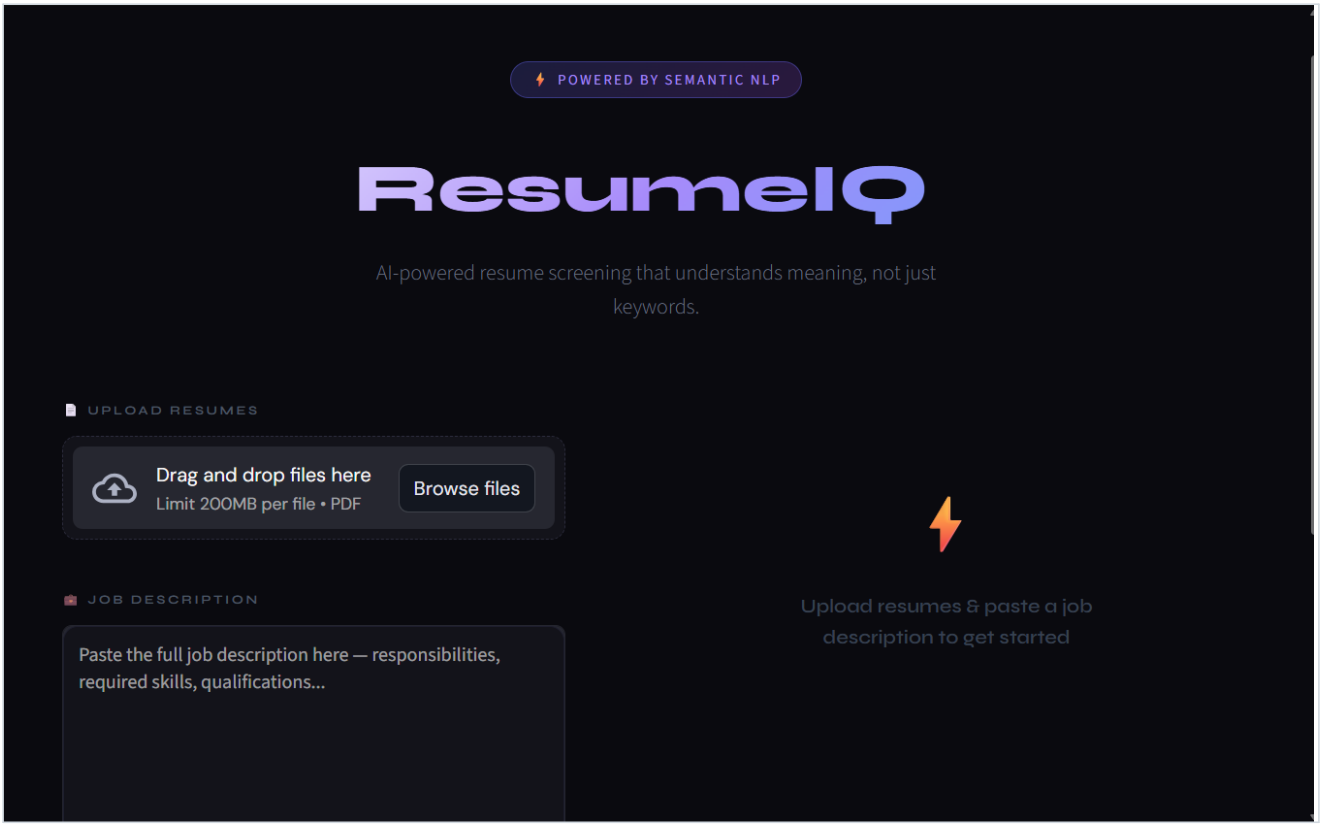
How It Works



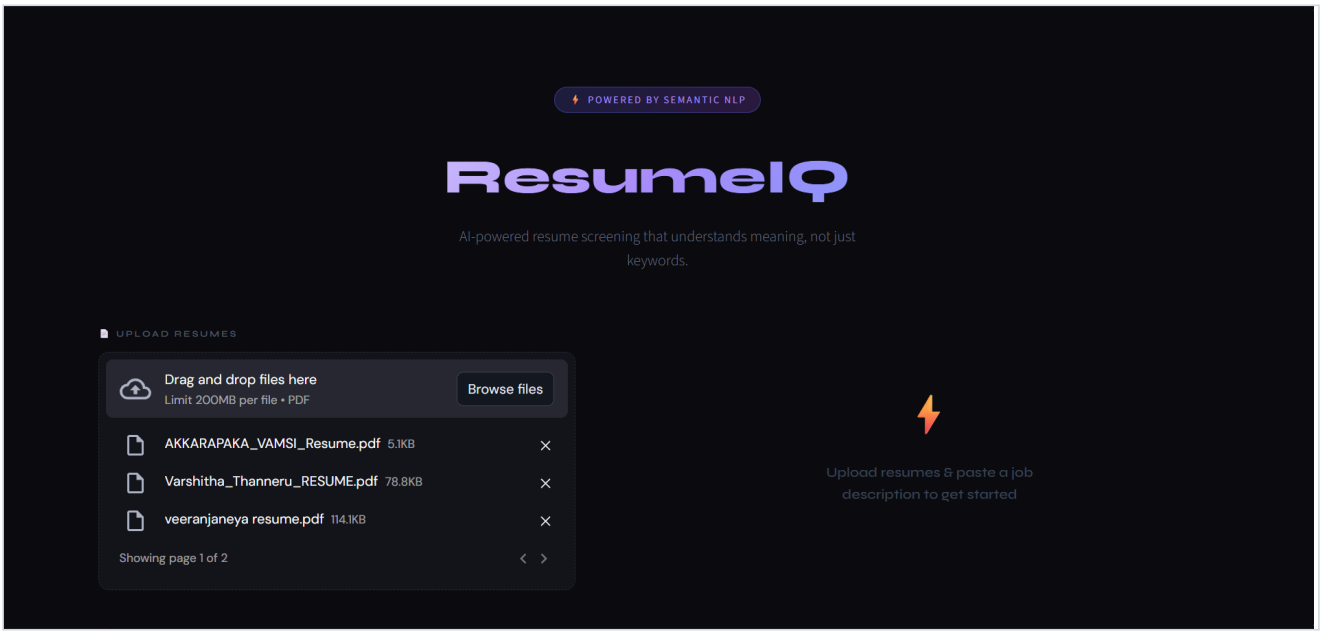
1. **Text Extraction** - PyPDF2 reads each page of the PDF and extracts raw text
2. **Text Cleaning** - Removes special characters, normalizes whitespace
3. **Embedding Generation** - `all-MiniLM-L6-v2` converts text to 384-dimensional vectors
4. **Cosine Similarity** - Measures the angle between vectors (1.0 = identical meaning)
5. **Skill Analysis** - Keyword extraction identifies matched and missing skills

Screenshots

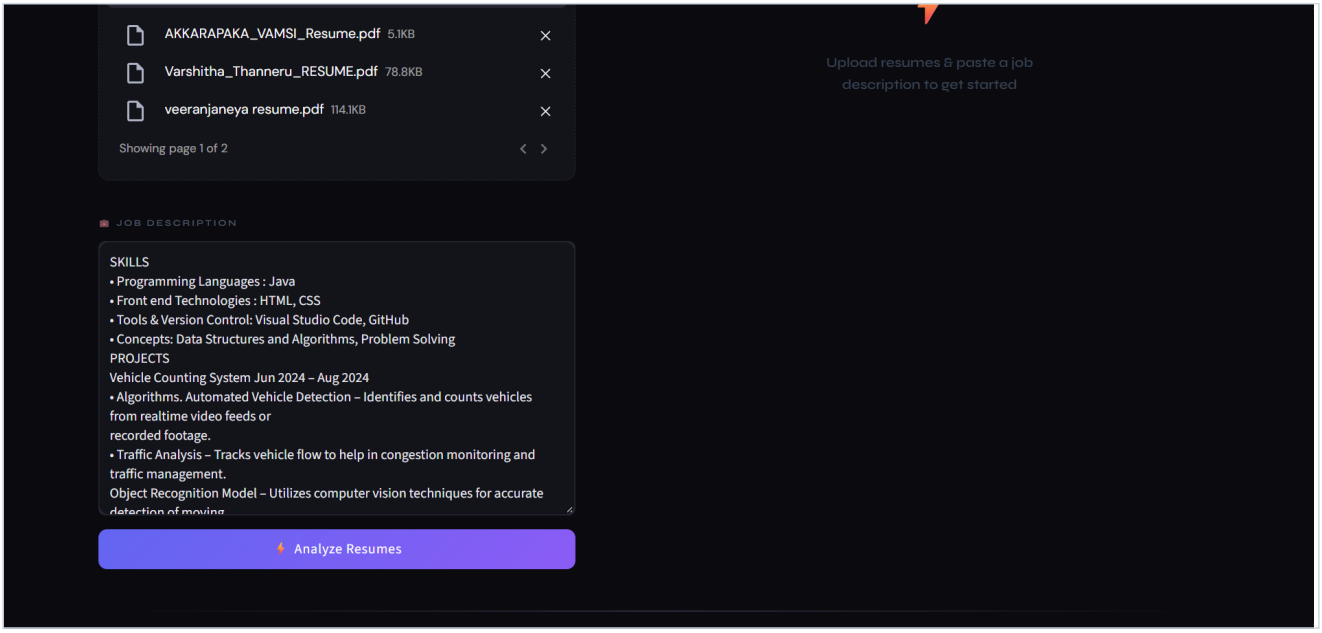
1. Home screen - upload resumes and paste a job description



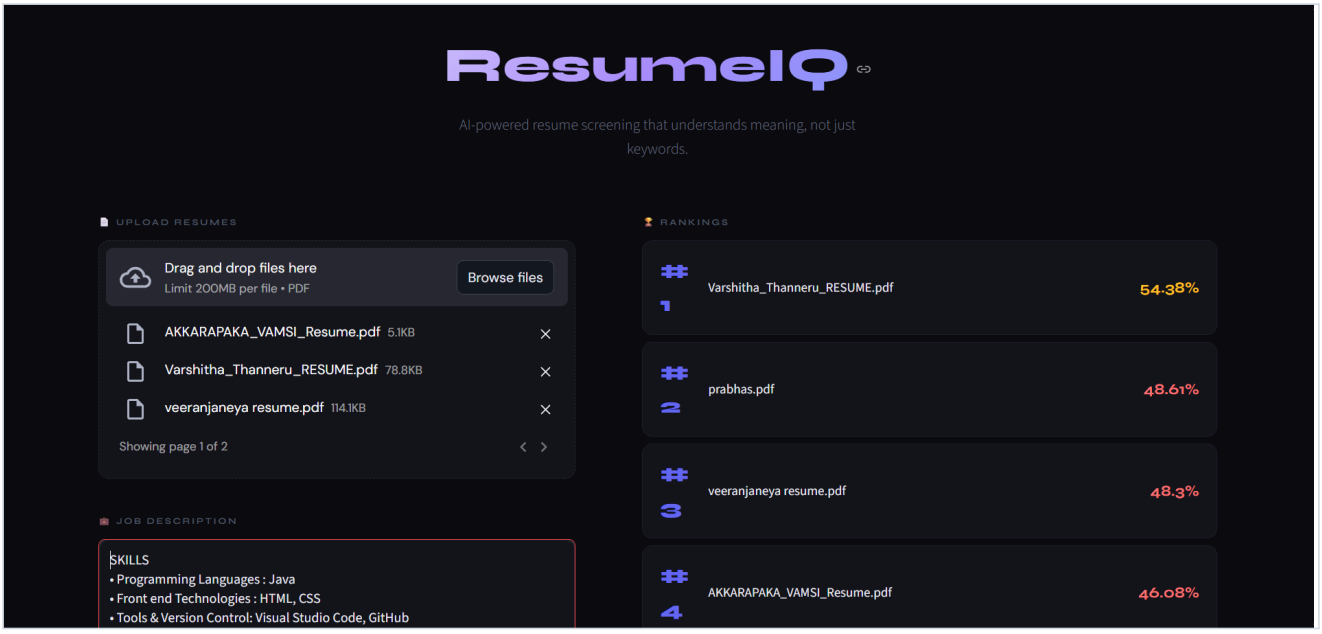
2. Multiple PDF resumes uploaded



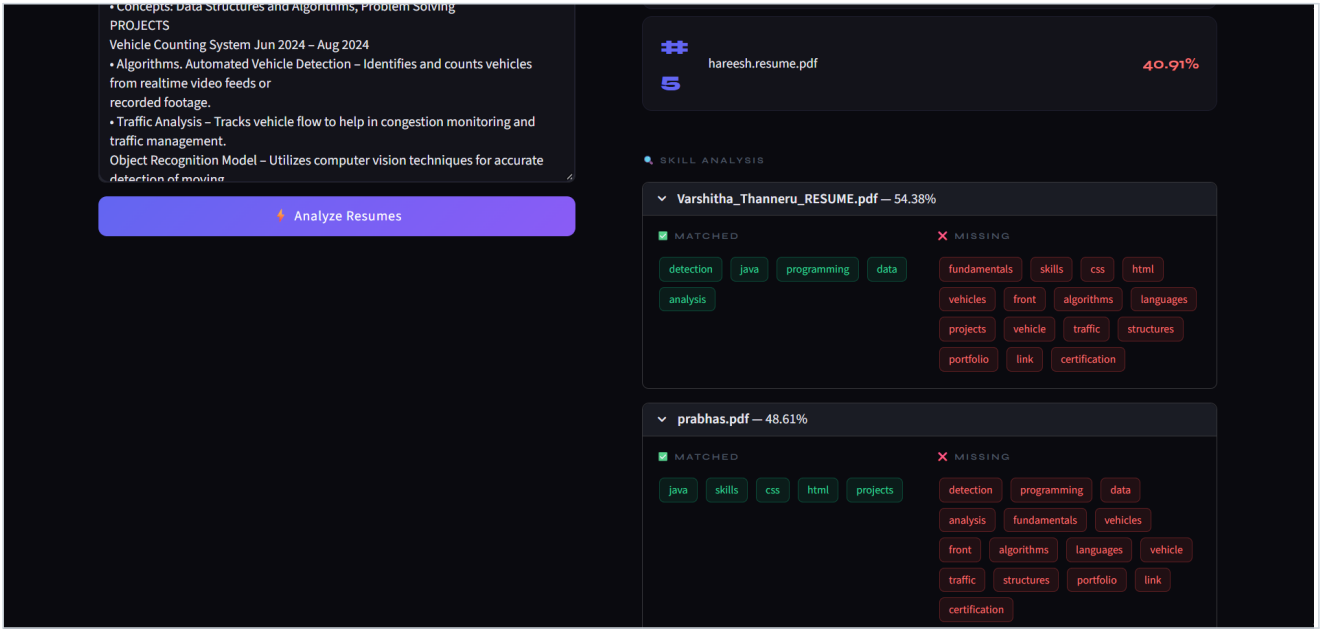
3. Job description entered, ready to analyze



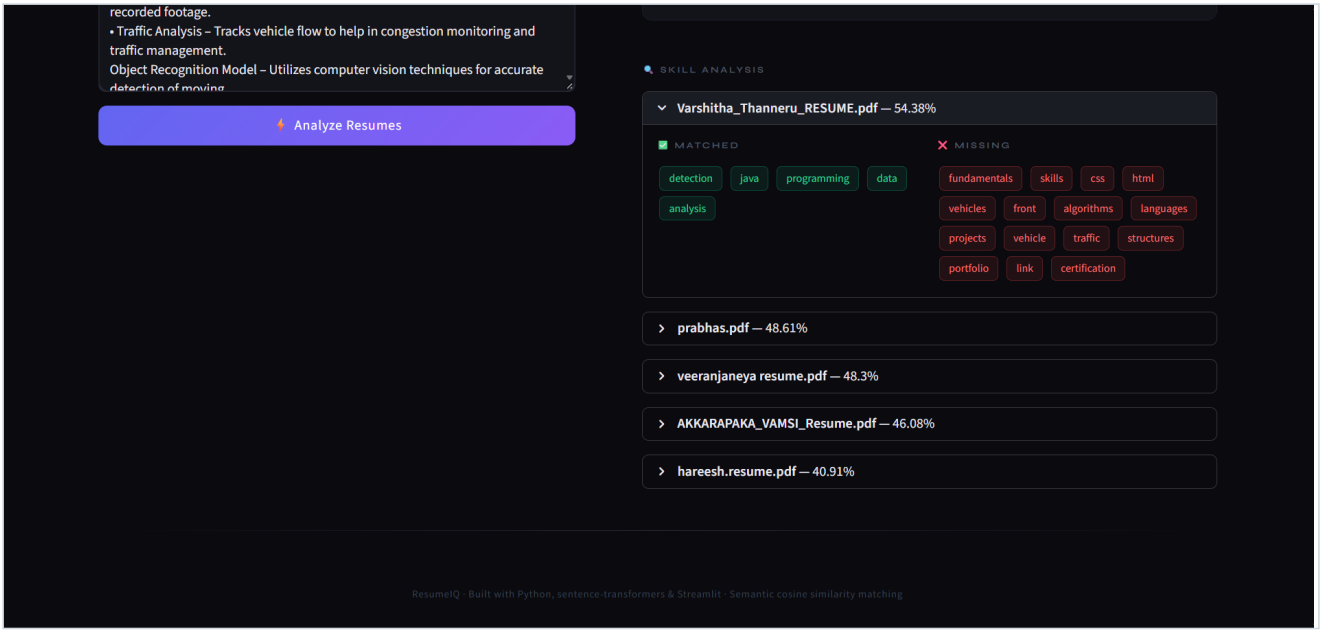
4. Candidate rankings by semantic match score



5. Skill analysis - matched and missing skills per resume



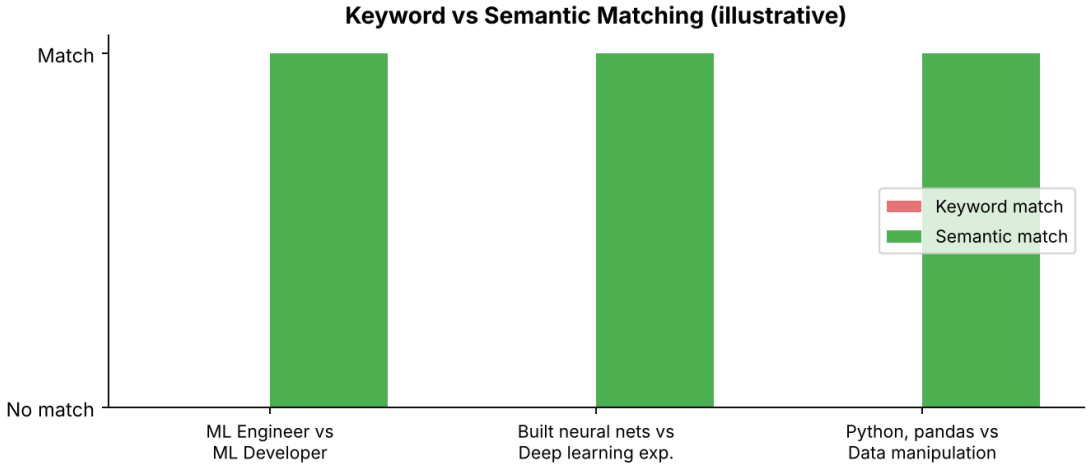
6. Expandable results for every uploaded resume



Why Semantic Matching?

Traditional keyword matching misses context:

Resume Says	Job Requires	Keyword Match	Semantic Match
ML Engineer	Machine Learning Developer	No	Yes
Built neural networks	Deep learning experience	No	Yes
Python, pandas	Data manipulation skills	No	Yes



Semantic matching using transformer embeddings captures **meaning**, not just exact words.

Future Improvements

- [ ] Deploy to Streamlit Cloud for public access
- [ ] Add support for DOCX resumes
- [ ] LLM-generated improvement suggestions per resume
- [ ] Batch processing via CSV upload
- [ ] API endpoint with FastAPI

Author

Brunda Venkatesh
M.S. Computer Science - AI/ML
LinkedIn: linkedin.com/in/brunda-venkatesh | GitHub: github.com/BrundaVenkatesh01

Thank You

ResumeIQ - AI-Powered Resume Screener

Semantic NLP matching that understands meaning, not just keywords.

Python | Streamlit | sentence-transformers | scikit-learn